

Experiment: Logisim Circuit Diagram Simulation from Boolean Expressions

Objective

The objective of this experiment is to design, implement, and simulate digital logic circuits corresponding to a given set of Boolean expressions using **Logisim Evolution** software. Each Boolean expression is realized using appropriate combinations of basic and derived logic gates, and the functional correctness of every designed circuit is verified against its theoretical truth table through simulation.

Theory

Boolean Algebra

Boolean algebra is a branch of mathematics that deals with binary variables and logical operations. It provides the mathematical foundation for the analysis and design of digital logic circuits. A Boolean variable can take only one of two values, **0 (LOW)** or **1 (HIGH)**, and Boolean expressions are formed by combining these variables using the fundamental operators **AND (·)**, **OR (+)**, and **NOT (complement)**. Boolean algebra also provides a set of laws and theorems, such as the commutative, associative, distributive, and De Morgan's theorems, which are used to simplify complex logical expressions before hardware implementation.

Logic Gates

Logic gates are the elementary building blocks of digital circuits. Each gate performs a specific logical operation on one or more binary inputs to produce a single binary output. The basic gates are **AND**, **OR**, and **NOT**, while **NAND**, **NOR**, **XOR**, and **XNOR** are derived gates formed by combining the basic gates. Logic gates are physically implemented using transistors, but for design and simulation purposes, they are represented using standard logic symbols.

Logic Circuit Implementation

Logic circuit implementation refers to the process of converting a Boolean expression into a physical or simulated arrangement of interconnected logic gates that produces the desired output for every possible combination of inputs. The implementation process involves identifying the required gates from the Boolean expression, connecting the input variables to the appropriate gates, and cascading the gate outputs to obtain the final output function.

SOP (Sum of Products)

The **Sum of Products (SOP)** form is a method of expressing a Boolean function as the logical OR (sum) of one or more AND (product) terms. Each product term consists of variables in either their true or complemented form. The SOP form is particularly convenient for circuit implementation, as it can be directly realized using a layer of AND gates followed by a single OR gate.

$$F_{SOP} = A \cdot B + \overline{A} \cdot C + A \cdot \overline{B} \cdot D$$

POS (Product of Sums)

The **Product of Sums (POS)** form is a method of expressing a Boolean function as the logical AND (product) of one or more OR (sum) terms. Each sum term consists of variables in either their true or complemented form. The POS form is directly realized using a layer of OR gates followed by a single AND gate.

$$F_{POS} = (A + B) \cdot (\bar{A} + C) \cdot (A + \bar{B} + D)$$

Why Logisim Is Used

Logisim Evolution is an open-source, academic digital logic simulator that allows the graphical design, construction, and simulation of digital circuits. Logisim is used in this experiment because it provides an intuitive drag-and-drop interface for placing logic gates, input/output pins, and wiring connections, and it allows real-time simulation of circuit behavior without requiring physical hardware. It also enables truth table generation, waveform analysis, and error detection, making it a suitable tool for verifying Boolean expressions before actual hardware implementation.

Advantages of Simulation Before Hardware Implementation

Simulating a digital logic circuit before physical implementation offers several engineering advantages:

- It allows verification of the logical correctness of a circuit without the risk of damaging physical components.
- It reduces the overall cost and time required for hardware prototyping.
- Design errors and wiring mistakes can be identified and corrected early in the design process.
- Multiple design iterations can be tested rapidly under different input conditions.
- It provides a safe environment for students and engineers to understand circuit behavior before working with actual ICs and breadboards.

Required Software

- Logisim Evolution
- Computer

Logic Gate Symbols

The standard logic gate symbols used in this experiment are summarized below.

Gate Name	Symbol	Boolean Expression
AND	> <i>Insert AND gate symbol here</i>	$Y = A \cdot B$
OR	> <i>Insert OR gate symbol here</i>	$Y = A + B$
NOT	> <i>Insert NOT gate symbol here</i>	$Y = \bar{A}$
NAND	> <i>Insert NAND gate symbol here</i>	$Y = \overline{A \cdot B}$

Gate Name	Symbol	Boolean Expression
NOR	> <i>Insert NOR gate symbol here</i>	$Y = \overline{A + B}$
XOR	> <i>Insert XOR gate symbol here</i>	$Y = A \oplus B$
XNOR	> <i>Insert XNOR gate symbol here</i>	$Y = \overline{A \oplus B}$

Experimental Procedure

The following procedure was followed to implement and simulate each of the five Boolean expressions in Logisim Evolution.

1. Logisim Evolution was opened, and a new blank project was created.
2. The required number of input pins (A, B, C, D) was placed on the canvas from the *Wiring* toolbox.
3. A single output pin was placed to represent the final function, **F**.
4. The given Boolean expression was analyzed, and the required logic gates (AND, OR, NOT, NAND, NOR) were identified.
5. The identified logic gates were selected from the *Gates* toolbox and placed onto the canvas in accordance with the structure of the Boolean expression.
6. NOT gates were connected to the input pins wherever a complemented variable was required in the expression.
7. The primary AND/OR gates were wired according to each product or sum term present in the expression.
8. The outputs of the intermediate gates were connected to the subsequent gates to form the complete nested logic structure of the expression.
9. The final gate output was connected to the output pin, **F**.
10. The circuit connections were checked for correctness, and all wires were verified to ensure they matched the intended Boolean expression.
11. The **Poke Tool** was used to toggle the input values, and the resulting output was observed on the output pin.
12. The circuit was simulated for all possible input combinations, and the simulated output was compared with the theoretical truth table.
13. The above steps (1–12) were repeated for each of the five given Boolean expressions.
14. Screenshots of each completed circuit were captured and saved for documentation purposes.

Circuit Diagrams

Expression 1

$$F = AB + \overline{B}C + AC + \overline{A}B\overline{D}$$

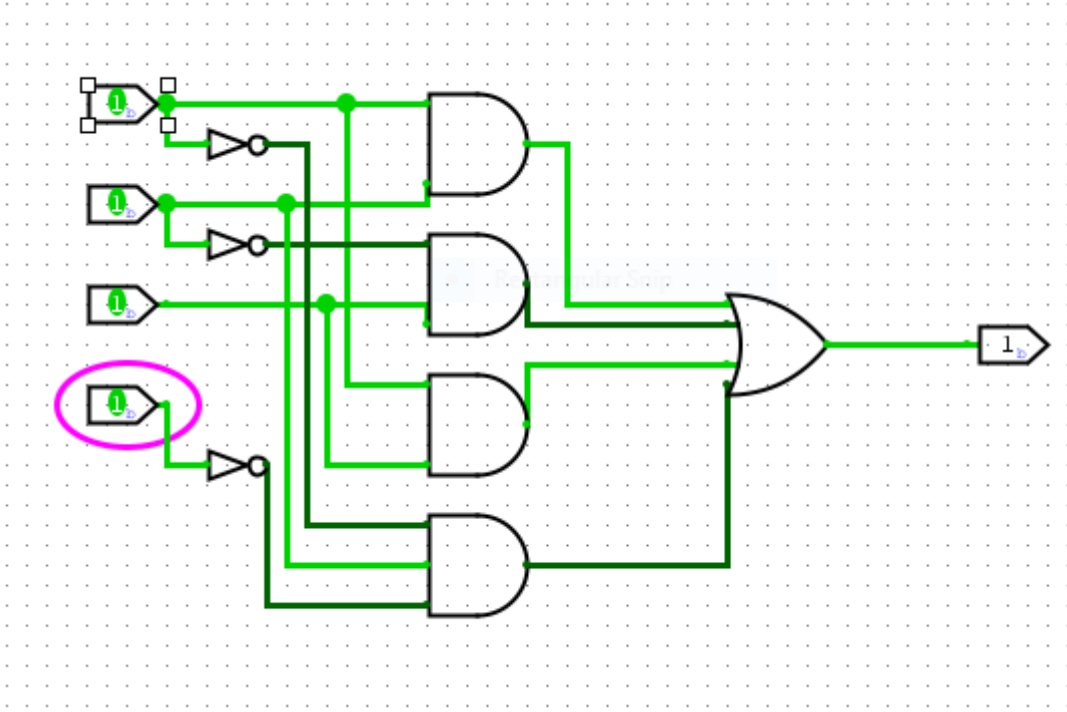


Figure 1: Logic circuit implementation of Expression 1 in Logisim Evolution.

Expression 2

$$F = [(\overline{A} + B)(C + \overline{D})] \cdot [A\overline{B} + \overline{C}D]$$

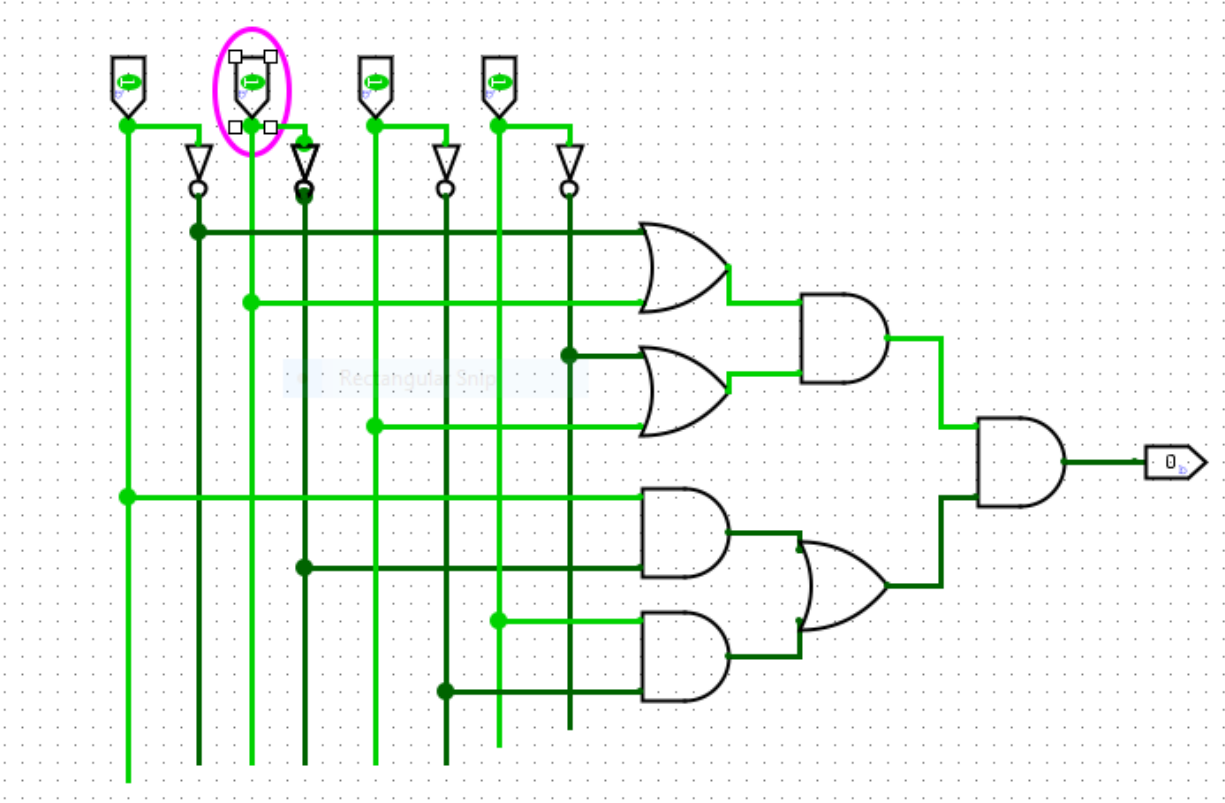


Figure 2: Logic circuit implementation of Expression 2 in Logisim Evolution.

Expression 3

$$F = ABCD + A\bar{B}CD + \bar{A}BCD + AB\bar{C}D + ABC\bar{D}$$

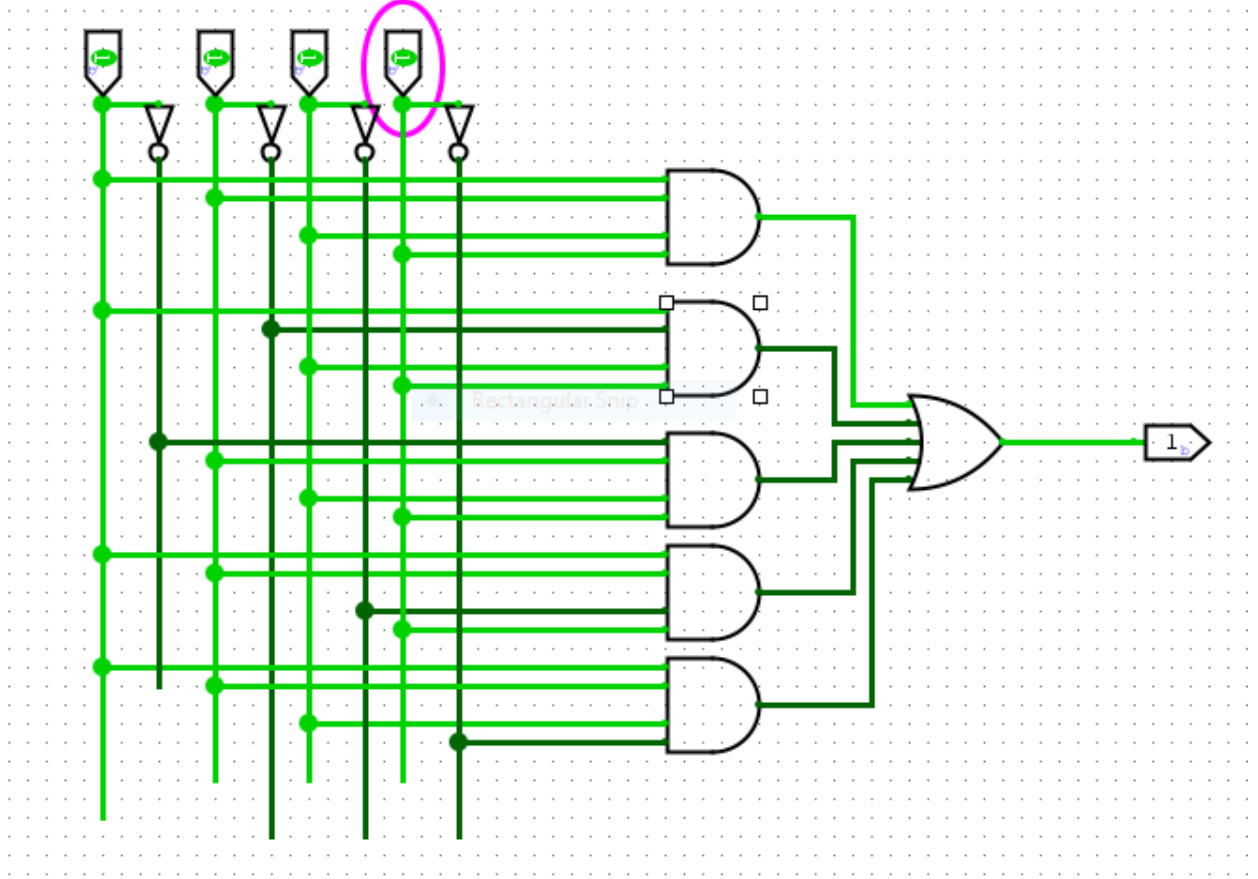


Figure 3: Logic circuit implementation of Expression 3 in Logisim Evolution.

Expression 4

$$F = (A + B + C + D)(A + B + \overline{C})(A + \overline{B} + C)(\overline{A} + B + C)$$

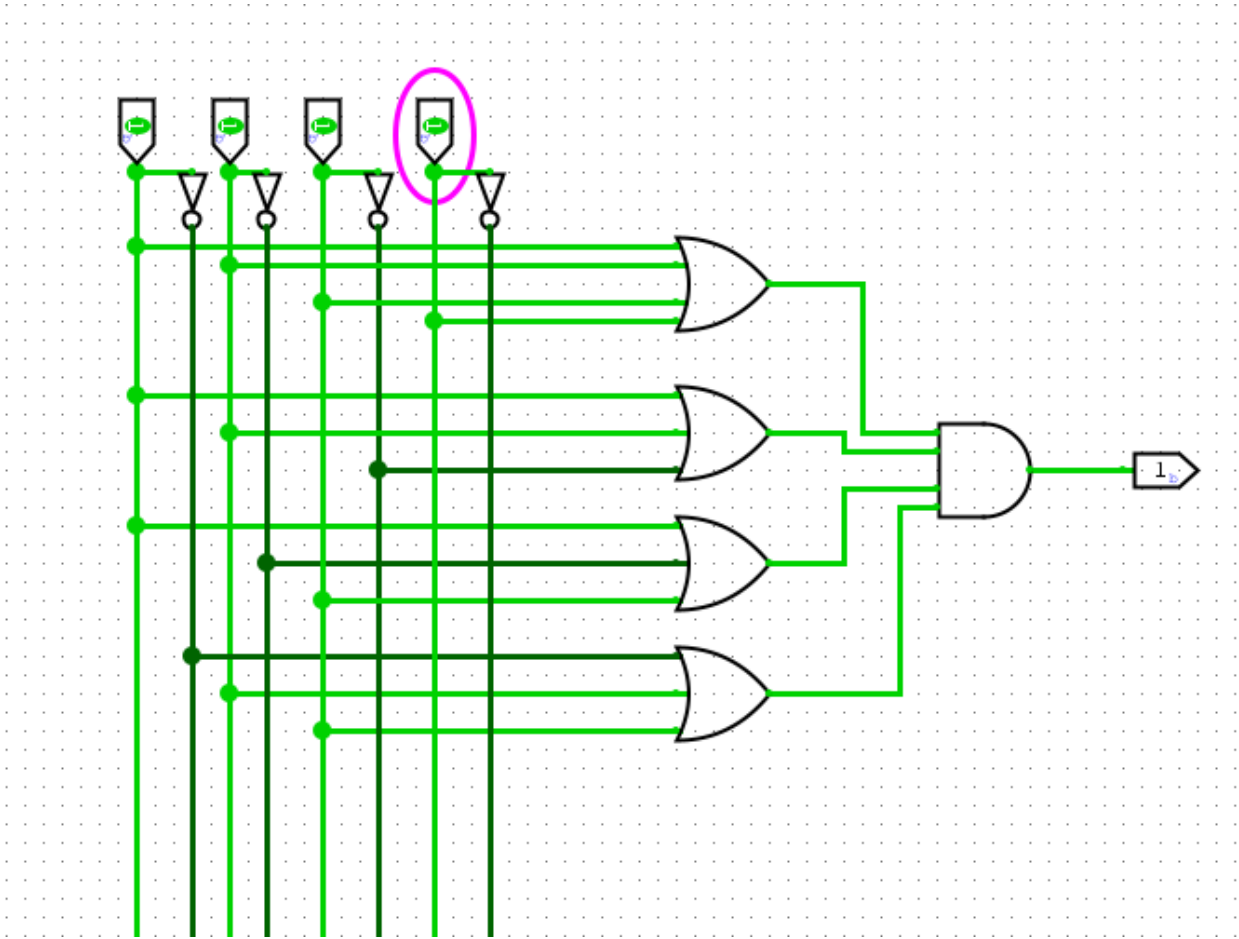


Figure 4: Logic circuit implementation of Expression 4 in Logisim Evolution.

Expression 5

$$F = [A + \overline{B}(C + D)] \cdot [\overline{A} + B(\overline{C} + \overline{D})]$$

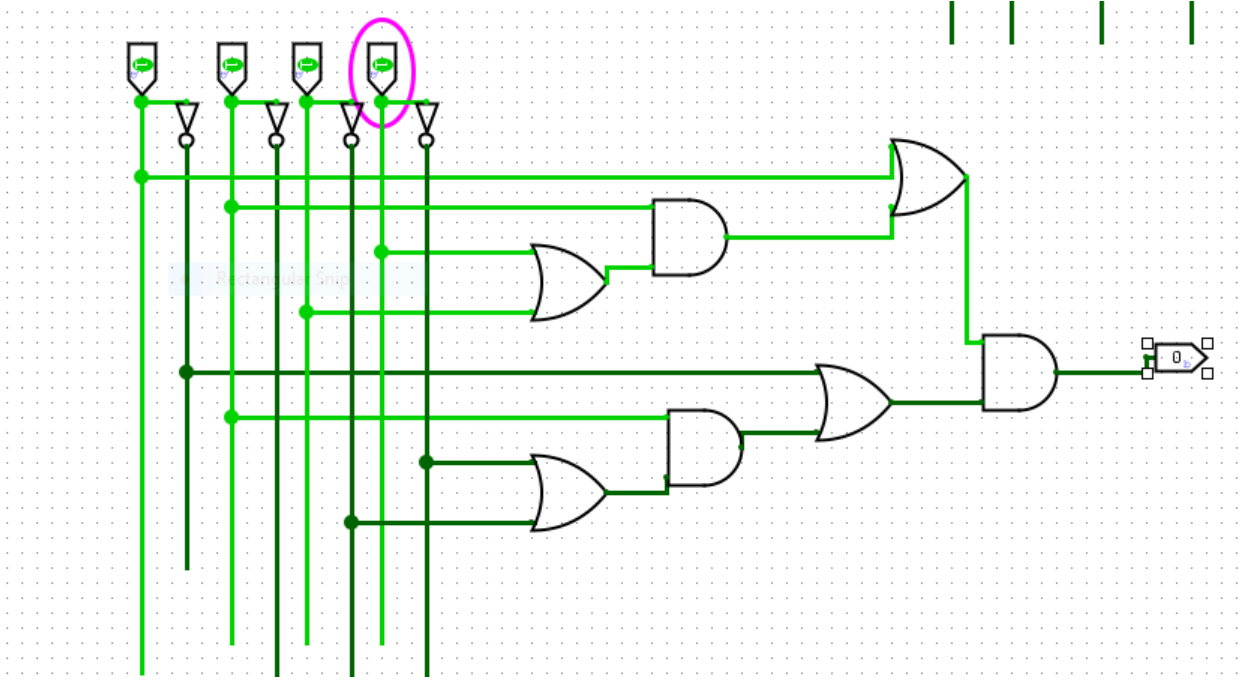


Figure 5: Logic circuit implementation of Expression 5 in Logisim Evolution.

Simulation

Each of the five circuits was simulated in Logisim Evolution using the following procedure. The four input pins (A, B, C, D) were toggled through all sixteen possible binary combinations, ranging from 0000 to 1111, using the Poke Tool. For every input combination, the resulting logic level at the output pin, **F**, was recorded as the **simulated output**. The simulated output was then compared against the **expected output**, calculated theoretically from the given Boolean expression. A circuit was considered correctly implemented only if the simulated output matched the expected output for all sixteen input combinations, thereby confirming agreement with the theoretical truth table.

Observation

Table 1: Observation for Expression 1 — $F = AB + \overline{B}C + AC + \overline{A}B\overline{D}$

A	B	C	D	Expected Output	Simulated Output
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	1	1
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0

A	B	C	D	Expected Output	Simulated Output
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

Table 2: Observation for Expression 2 — $F = [(\bar{A} + B)(C + \bar{D})] \cdot [A\bar{B} + \bar{C}D]$

A	B	C	D	Expected Output	Simulated Output
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1	1	0	0

Table 3: Observation for Expression 3 — $F = ABCD + \bar{A}\bar{B}CD + \bar{A}BCD + A\bar{B}\bar{C}D + ABC\bar{D}$

A	B	C	D	Expected Output	Simulated Output
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	0	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	1	1
1	1	0	0	0	0

A	B	C	D	Expected Output	Simulated Output
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

Table 4: Observation for Expression 4 — $F = (A+B+C+D)(A+B+\overline{C})(A+\overline{B}+C)(\overline{A}+B+C)$

A	B	C	D	Expected Output	Simulated Output
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

Table 5: Observation for Expression 5 — $F = [A + \overline{B}(C + D)] \cdot [\overline{A} + B(\overline{C} + \overline{D})]$

A	B	C	D	Expected Output	Simulated Output
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	0	0

Discussion

The simulated output obtained for each of the five circuits in Logisim Evolution matched exactly with the theoretically expected output derived from the corresponding Boolean expression, confirming that all circuits were implemented correctly. It is worth noting that **Expression 2** simplifies identically to logic **0** for every possible combination of A, B, C, and D; this is a valid Boolean algebra result, since the two AND-ed bracketed terms in the expression are mutually exclusive and can never be logic **HIGH** simultaneously. This case highlights the importance of algebraic simplification prior to hardware implementation, as it prevents unnecessary construction of a circuit that produces a constant, non-functional output.

Common mistakes encountered during circuit construction included incorrect placement of NOT gates on the wrong variable, swapped input connections at the AND/OR gate terminals, and unintentionally floating (unconnected) input pins, all of which produced incorrect simulation results until corrected. Careful cross-verification of each gate connection against the original Boolean expression was necessary to resolve these errors.

Gate minimization is of significant importance in digital circuit design, as it directly affects the number of gates required, the propagation delay, and the overall power consumption and cost of the final hardware implementation. By simplifying a Boolean expression using Boolean algebra or Karnaugh maps prior to circuit implementation, redundant gates can be eliminated, resulting in a more efficient and economical circuit design without altering its logical functionality.

Result

All five given Boolean expressions were successfully implemented as digital logic circuits and verified through simulation using **Logisim Evolution**. The simulated output of each circuit was found to be in complete agreement with its theoretically expected truth table, confirming the correctness of the designed logic circuits.

Precautions

1. All logic gates must be selected carefully according to the exact operators present in the Boolean expression.
2. Input and output pins must be labeled clearly to avoid confusion during simulation.
3. All wire connections should be checked thoroughly to prevent floating or short-circuited nodes.
4. NOT gates must be placed on the correct variable to ensure the correct complemented input is used.
5. The circuit should be simulated for all possible input combinations, not just a few, to fully verify functional correctness.
6. Gate orientations and pin numbering should be verified before finalizing the wiring.
7. The project file should be saved periodically to prevent loss of work during the design process.
8. Screenshots of completed circuits should be captured with all input/output labels clearly visible for documentation purposes.

Conclusion

In this experiment, five distinct Boolean expressions were successfully translated into digital logic circuits and simulated using Logisim Evolution. The process reinforced the practical understanding of Boolean algebra, SOP and POS representations, and the systematic procedure of converting a

mathematical logic expression into an interconnected arrangement of logic gates. The complete agreement between the expected and simulated truth tables for all five expressions validates both the theoretical Boolean analysis and the accuracy of the Logisim-based implementation. This experiment demonstrates the practical value of software-based simulation as a reliable and cost-effective verification step prior to physical hardware implementation of digital logic circuits.